Large-Scale Legacy Systems Modernization: An Experience Report on Transition Architectures and AI in a Brazilian Bank*

Manoel Valerio da Silveira Neto[1,2] and Bruno Pereira Soares[1]

[1] Banco Bradesco SA, Brasil
bruno.p.soares@bradesco.com.br
[2] Pontifícia Universidade Católica do Paraná (PUCPR), Curitiba, Brasil
manoel.silveira@pucpr.edu.br

Abstract. The modernization of legacy systems is no longer a one-off project but has become an ongoing program in organizations with critical systems. This experience report describes the evolution of a Brazilian bank from a pilot to a large-scale program, combining agile practices, transition architectures, and the systematic use of AI to support analysis, automation, and decision-making. We present a lightweight phased framework, the standardization of architectural evidence (AS-IS/Transition/TO-BE), and the reduction of variability between squads through governance, enablement, and reusable capabilities. The results include an estimated 65% reduction in analysis effort through reverse engineering automation and a prioritized modernization scope of 199 business services managed across multiple waves.

Keywords: Legacy Systems Modernization · Transition Architecture · Modernization Framework · Banking Architecture · Artificial Intelligence

1 Introduction

In banks, legacy is rarely just old software: it concentrates business rules, system dependencies, and operational costs. As competitiveness demands shorter cycles, it amplifies coupling, technical debt, and low observability. In this scenario, we launched a corporate modernization program to reduce mainframe dependence, accelerate delivery, and preserve operational continuity.

We structured the initiative based on the Banking Industry Architecture Reference Model (BIAN) [5], adapted to the institution's services, as a reference to connect initiatives to business capabilities. Execution combined agile cycles inspired by the *Spotify Model* [17,16] and principles from *Team Topologies* [24], treating digital transformation as a cultural, technological, and operational phenomenon [26,25].

* This paper was submitted to the Experience Reports track of XP 2026, the 27th International Conference on Agile Software Development, held in São Paulo, Brazil, from April 8–11, 2026.

This report starts from a pilot (V0) with 14 squads working on mainframe systems and cloud-native solutions on Microsoft Azure. V0 showed that scale increases inconsistency: similar decisions were made in different ways, incremental migrations prolonged coexistence, and the lack of documentation increased the reverse-engineering effort. Based on these lessons, we evolved to V1 with (i) phases and templates to make the work repeatable, (ii) transition architectures (CDC, Strangler Fig, anti-corruption layers) for safe coexistence, and (iii) systematic use of AI to accelerate analysis and automation.

Prior work contextualizes challenges in architectural refactoring, migration to event-driven architectures, and the use of AI in software evolution and modernization [27,18,20], as well as estimates of potential AI gains[3] . Our focus is the how and the why: how we combined these approaches under real-world constraints and which lessons can be reused by other organizations.

2 Context of the Report and How We Collected Evidence

What we describe was experienced by us in collaboration with squads, architects, and leadership. This paper is an *experience report* [13] and aims to extract practical lessons from decisions, trade-offs, and observed results [6]. To maintain descriptive rigor, we triangulated participant observation, recurring conversations, and review of program artifacts.

Two organizational elements sustained execution: (i) a dedicated **Tech Modernization** team, responsible for governance (criteria, templates, support for recurring decisions, adherence to phases, and minimum evidence); and (ii) the **Special Products** team, organized by the IT Architecture area, to **enable reusable capabilities** (e.g., CDC, Kafka-based messaging, and file transfer mechanisms) through "platform products", blueprints, components, and expert support, reducing entry cost and variability across squads.

Team topology and delivery cadence: Modernization execution was organized by business units (BUs), tribes, and squads, with one or more solution architects per BU/tribe and squads typically operating with about six engineers in sprint-based agile delivery. This distributed modernization work across products while keeping architectural support close to delivery teams. As public scale context, Bradesco reported more than 10.5 thousand technology professionals in 2025 and 82,095 employees at year-end [**?**,4].

The narrative follows the program sequence (diagnosis, V1 design, intervention, and evaluation), and evidence was triangulated from recurring conversations with the 14 V0 squads, observation in rituals/forums, and framework artifacts (templates, decisions, diagrams, and transition records), synthesized via thematic coding.

The selection of the 14 squads followed corporate criteria: low functional complexity and mainframe execution costs high enough to justify prioritization. Participants included senior architects, specialists, architecture managers, and

[3] https://www.mckinsey.com/capabilities/quantumblack/our-insights/ai-for-it-modernization-faster-cheaper-and-better

Table 1. V1 six-phase framework (compact summary).

Phase	Main purpose / output
Discovery & Assessment	Inventory legacy assets, dependencies, risks, and initial backlog.
Functional Cutout	Define slices, prioritize by value/risk, and record 6Rs rationale.
Transition Architecture	Define AS-IS / Transition / TO-BE and coexistence strategy.
Implementation	Build increments with validation and decision traceability.
Cutover & Rollout	Execute activation, monitoring, and rollback criteria.
Shutdown	Decommission legacy functionality and close compliance evidence.

software engineering managers, enabling an integrated view of technical, organizational, and operational challenges.

Because this work is conducted in an industrial setting, confidentiality constraints shape what can be disclosed. We therefore report only non-sensitive metrics and focus on mechanisms, decision rationales, and observable effects that can be generalized without exposing proprietary details. To reduce interpretive bias, we contrasted perspectives across roles (architecture, engineering, and leadership) and used the artifacts produced by the framework (templates, diagrams, transition records, and decision rationales) as stable evidence to support the narrative and help readers understand which evidence sources can make similar experience reports more credible under industrial constraints.

3 The Big Picture: From Pilot to Framework V1

From V0 to V1, the main shift was not only technical but operational: V0 exposed recurring problems (legacy understanding, coexistence, and repeated decisions), while V1 introduced a repeatable framework (phases, templates, transition architectures, decision support, and AI) to reduce squad-level divergence. For practitioners, the key takeaway is that scaling modernization requires turning recurrent trade-offs into shared mechanisms and comparable evidence, while keeping operational support (e.g., TMS) as a support layer for V1 rather than a new framework version.

As a generic example, a hypothetical business service (e.g., *Customer Profile*) moves from dependency/slice definition (Functional Cutout) to coexistence design (e.g., CDC plus Strangler-style routing, with rollback and observability criteria) and then to incremental implementation, cutover, and shutdown with validation/compliance evidence. This illustrates the framework without exposing proprietary details.

3.1 Transition Architectures: Coexisting Comes at a Price

The transition from legacy to modern systems requires security, integration, and operational continuity. CDC (Change Data Capture) [15] was central to coexistence by replicating changes from legacy databases to modern ones, enabling synchronization and progressive data migration. However, CDC is not magic:

it shifts costs from development to modeling, governance, auditing, and observability.

We treated CDC, messaging, and file transfer as reusable *enablers* (rather than isolated project decisions), supported by the Special Products team (components, patterns, and guides), reducing rework and variability. In parallel, governance required comparable decisions across projects by standardizing three architectural views: AS-IS, TO-BE, and Transition. These views were modeled using the C4 Model [7] and stored as evidence in a corporate architecture repository based on Ardoq [2], reducing ambiguity and accelerating reviews, especially when coexistence extends across multiple releases.

To reduce risk, we adopted a context-driven set of patterns, including Strangler Fig; Proxy and Delegate for interception/redirection; Anti-Corruption Layer for legacy-modern translation; EDA, CQRS, and Event Sourcing for integration and consistency where needed; resilience patterns (e.g., Circuit Breaker, Saga, Bulkhead, Outbox, Sidecar); and CDC for legacy-modern data synchronization [12,9,8,23,14,11,10,22,15].

The mindset shift in V1 was to move away from an "infinite catalog" and make trade-offs explicit per context (e.g., CDC-based coexistence requires minimum observability and consistency criteria), reducing preference-driven decisions across squads.

3.2 Use of Artificial Intelligence: Accelerator, But Only with Guardrails

AI was integrated across framework phases to address two recurring bottlenecks: legacy understanding and repetitive engineering tasks. Prior work highlights these opportunities in modernization [20]; here we report where AI helped and which safeguards were necessary.

We applied AI in five areas: code/data analysis (technical debt, vulnerabilities, dependencies, refactoring opportunities), code generation/refactoring (e.g., COBOL-to-Java support and microservice scaffolding [21]), decision support and prioritization (including 6Rs), reverse-engineered documentation/diagrams, and database understanding (e.g., data dictionaries and textual ER representations).

The gains were sustainable only with human review and governance. During implementation, for example, AI sped up API drafts and Java code aligned with Clean Architecture [19], but required checklists and validation; without guardrails, subtle inconsistencies with legacy business rules emerged.

To operationalize AI safely, we adopted lightweight guardrails: human-in-the-loop reviews, checklists for functional equivalence and security/privacy constraints, and traceability of AI-assisted outputs linked to phase artifacts; acceptance remained conditional on conventional validation and explicit sign-off for changes affecting coexistence or data consistency.

3.3 Decision Trees: Standardize to Scale

Decision trees were used to standardize recurrent technology choices by making criteria explicit and traceable (e.g., Azure SQL vs. MongoDB for persistence, and Azure Databricks vs. Spring Batch for data processing, depending on volume, latency, integration, and operational constraints). Even lightweight trees reduced rework and onboarding friction by documenting why an option was selected in a given migration context.

3.4 Operationalization and Scale: Governance, Enablement, and a Management System

V0 showed that scaling depends more on operationalization than on process design. Tech Modernization governance reduced variation, step-by-step videos standardized onboarding, and Special Products reduced technical friction through reusable capabilities, allowing squads to focus on functional scoping and incremental migration.

As the number of initiatives increased, traceability became a bottleneck. We refer to the support system under development as the Tech Modernization System (TMS), which records phases, status, costs, MIPS reduction, architectural evidence (AS-IS/Transition/TO-BE in C4 and/or Ardoq), and phase input/output artifacts, improving auditability, traceability, and transparency in V1 without changing the framework phases, which can help practitioners scale governance without redesigning the framework itself.

4 Results Observed and Lessons Learned

Due to institutional constraints, sensitive metrics cannot be disclosed. Even so, we observed consistent qualitative effects: greater predictability and governance; more consistent architectural evolution across squads; reduced manual effort through automation; better integration between agile practices and architecture; and AI support both in reducing technical debt and in understanding business rules in mainframe systems. We also observed incremental migration of core applications using CDC for temporary coexistence, reducing mainframe processing consumption, and the adoption of replatforming for less critical applications.

4.1 Non-Confidential Quantitative Indicators

Two non-confidential indicators were authorized for publication. First, the reported 65% value represents an estimated reduction in analysis effort ("analysis performance") for producing a minimum analysis package for a service slice (dependencies, interface/data understanding, and initial decision-support documentation) with AI-assisted reverse engineering/documentation versus a manual baseline. This 65% value is an aggregated estimate based on specialist judgment across modernization cases; it is narrower than bank-wide productivity indicators publicly reported by the institution (e.g., +118% productivity in 2025[3]),

because it measures a specific modernization-analysis activity rather than overall technology productivity. To respect confidentiality constraints, we omit raw time logs, service identifiers, and per-squad breakdowns.

Second, the reported scope (199 business services) was obtained through a strategic prioritization over a portfolio of 3,000 business services (BIAN-inspired service catalog/capability mapping), using customer impact, mainframe MIPS consumption, technological obsolescence, maintenance cost, and infrastructure cost, with a Pareto-oriented logic (80/20). These 199 services were prioritized for modernization and managed across multiple waves under the same governance model (phases, templates, and minimum evidence requirements).

These figures contextualize scale and practical impact; next, we synthesize practitioner-oriented lessons for large-scale modernization.

4.2 Lessons (L1–L8): what readers can expect and apply in large-scale modernization

The lessons below summarize practitioner-oriented guidance for large-scale legacy modernization under operational and confidentiality constraints.

(L1) Plan for coexistence costs, not only coexistence mechanisms. CDC and translation layers reduce *big-bang* risk, but they also introduce governance overhead (data modeling, monitoring, and auditing) that should be planned explicitly [15].

(L2) Standardize decisions and evidence to reduce divergence across squads. Templates, decision trees, and comparable AS-IS/Transition/TO-BE views (C4/Ardoq) improve review speed and reduce rework caused by inconsistent documentation [1].

(L3) Use AI as an accelerator with guardrails. AI gains were more sustainable with human review, explicit quality criteria, and governance checkpoints; otherwise, speed gains were offset by rework [19].

(L4) Organizational maturity affects speed as much as technical choices. Practices inspired by Spotify and Team Topologies improve coordination, but enablement and onboarding remain essential to reduce execution variability [17,16,24].

(L5) Incremental transition improves flexibility but increases operational complexity. Patterns such as Strangler, Proxy, Delegate, and EDA support progressive migration, but prolonged coexistence demands stronger observability and resilience [14,22].

(L6) Functional scoping is a socio-technical decision. Effective prioritization combines technical feasibility with operational risk, regulatory impact, and value flow, not only engineering preferences.

(L7) Scaling a framework requires traceability mechanisms, not only guidance artifacts. As initiatives increased, the TMS became necessary to support V1 operationalization by recording phases, costs, MIPS reduction, and transition evidence without changing the framework phases.

(L8) Reusable capabilities reduce adoption cost and improve consistency. An enabling team (Special Products) accelerated execution by provid-

ing reusable components and blueprints (e.g., CDC, Kafka, file transfer) instead of per-squad reinvention.

5 Conclusion

We report the evolution from a pilot to a large-scale legacy modernization program in a Brazilian bank, combining agile practices, transition architectures, and the systematic use of AI. The central lesson was to make modernization repeatable: turning legacy uncertainty into traceable decisions, reducing variation across squads, and treating coexistence as an explicit cost.

Despite limitations on publishable metrics, we observed gains in governance and architectural consistency, including an estimated 65% reduction in analysis effort and a prioritized modernization scope of 199 business services across multiple waves. The lessons highlight that AI delivers impact with guardrails, and that transition architectures enable incremental migration while increasing operational complexity. We hope these findings help other organizations reduce risk and rework when modernizing legacy systems at scale.

Threats and reporting limitations: This is a single-bank experience report; observations by involved participants may introduce bias, mitigated through artifact review and contrasting perspectives across roles. Contractual constraints also limited disclosure of detailed metrics and architecture visualizations, so we report qualitative effects and non-confidential indicators, consistent with experience-report goals [6].

Acknowledgments

This study was supported by the Coordination for the Improvement of Higher Education Personnel – Brazil (CAPES) – Finance Code 001.

References

1. Amazon Web Services: Migrating to aws: Best practices and strategies. Tech. rep., AWS (2018), https://d1.awsstatic.com/whitepapers/Migration/aws-migration-whitepaper.pdf
2. Ardoq AS: Ardoq. https://www.ardoq.com/ (2025)
3. Banco Bradesco S.A.: 4t25 result presentation. https://www.bradescori.com.br/ (2026), investor presentation
4. Banco Bradesco S.A.: Economic and financial analysis report. https://www.bradescori.com.br/ (2026), investor presentation
5. Banking Industry Architecture Network: Bian business architecture framework. Tech. rep., BIAN (2019), disponível em: https://bian.org/. Acesso em: 28 nov. 2025
6. Bergman, L., Woda, H., Rodenbach, E., van Latum, F., van Solingen, R., Cinti, S., Oldano, G., Orazi, E., Gresse, C., Hoisl, B., Rombach, D., Ruhe, G., Oivo, M., Fuggetta, A., Lavazza, L., Morasca, S., Barbati, R., Judge, T.R., Williams,

A., Sabbatici, G., Rome, A., Panarono, P., Farina, S., Roald, H.M., Larsen, J.O., Powell, A., Brown, D., Eickelpasch, G.: Experience Reports, pp. 101–255. Springer Berlin Heidelberg, Berlin, Heidelberg (2001). https://doi.org/10.1007/978-3-642-56618-9_6, https://doi.org/10.1007/978-3-642-56618-9_6
7. Brown, S.: The c4 model for visualising software architecture. https://c4model.com/ (2015)
8. Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., Stal, M.: Pattern-Oriented Software Architecture: A System of Patterns. Wiley (1996)
9. Fowler, M.: Patterns of Enterprise Application Architecture. Addison-Wesley (2002)
10. Fowler, M.: Event sourcing (2005), https://martinfowler.com/eaaDev/EventSourcing.html
11. Fowler, M.: Cqrs (2011), https://martinfowler.com/bliki/CQRS.html
12. Gamma, E., Helm, R., Johnson, R., Vlissides, J.: Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, Reading, MA (1994)
13. Heitmeyer, C.L.: Introduction to the experience reports track. In: Proceedings of the 27th International Conference on Software Engineering (ICSE '05). p. 620. ACM (2005). https://doi.org/10.1145/1062455.1062541, https://dl.acm.org/doi/10.1145/1062455.1062541
14. Hohpe, G., Woolf, B.: Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Addison-Wesley (2003)
15. Inmon, W.H.: Building the Data Warehouse. Wiley Publishing, Indianapolis, 4 edn. (2005)
16. Kniberg, H.: Spotify engineering culture – part 2. https://blog.crisp.se/wp-content/uploads/2014/06/spotify-engineering-culture-part2.pdf (2014), accessed: 2025-11-24
17. Kniberg, H., Ivarsson, A.: Scaling agile @ spotify with tribes, squads, chapters & guilds. https://blog.crisp.se/wp-content/uploads/2012/11/SpotifyScaling.pdf (2012), accessed: 2025-11-24
18. Laigner, R., Kalinowski, M., Diniz, P., Barros, L., Cassino, C., Lemos, M., Arruda, D., Lifschitz, S., Zhou, Y.: From a monolithic big data system to a microservices event-driven architecture. p. 213 – 220 (2020). https://doi.org/10.1109/SEAA51224.2020.00045, https://www.scopus.com/inward/record.uri?eid=2-s2.0-85096582582&doi=10.1109%2fSEAA51224.2020.00045&partnerID=40&md5=7308c76d96df207c3ca9379a0f407a13, cited by: 45
19. Martin, R.C.: Clean Architecture: A Craftsman's Guide to Software Structure and Design (2017). https://doi.org/10.1177/1356389011400889
20. Melo, I., Polónia, D., Teixeira, L.: Human–ai collaboration in the modernization of cobol-based legacy systems: The case of the department of government efficiency (doge). Computers **14**(7) (2025). https://doi.org/10.3390/computers14070244, https://www.mdpi.com/2073-431X/14/7/244
21. Newman, S.: Building Microservices. O'Reilly, 2 edn. (2021)
22. Nygard, M.T.: Release It!: Design and Deploy Production-Ready Software. Pragmatic Bookshelf (2007)
23. Richardson, C.: Microservices Patterns: With examples in Java. Manning (2018)
24. Skelton, M., Pais, M.: Team Topologies: Organizing Business and Technology Teams for Fast Flow. IT Revolution Press (2019)
25. Vial, G.: Understanding digital transformation: A review and a research agenda. The Journal of Strategic Information Systems **28**(2-3), 118–144 (2019)

26. Westerman, G., Bonnet, D., McAfee, A.: Leading Digital: Turning Technology into Business Transformation. Harvard Business Review Press (2014)
27. Zimmermann, O.: Architectural refactoring: A task-centric view on software evolution. IEEE Software **32**(2), 26–29 (2015). https://doi.org/10.1109/MS.2015.37